

Figma Design RL Environment

Technical Report and Sample Submission

Prepared by the Metaphi Environments Team

1. Executive Summary

This report accompanies the source code for `metaphi/figma-design`, an RL environment for training and evaluating LLM agents on the production discipline of UI design in Figma. It is built around three pillars that together make it a practical training ground rather than a screenshot-grading benchmark:

1. **A Figma-shaped sandbox we own end-to-end.** Our **Figma Clone** speaks the Figma Plugin API surface and exposes the same Model Context Protocol (MCP) tools that real agents use against Figma Desktop today. Tasks run inside it without any dependency on Figma cloud, accounts, or rate limits.
2. **Real designs, not synthetic stubs.** Tasks are seeded from **real Figma files ported into the Figma Clone** by subject matter experts, so agents see the variables, components, auto-layout, styled text segments, and structural messiness of production design systems.
3. **A rubric-and-reward system designed for RL.** Every task ships an `eval-spec.json` authored per task by subject matter experts. The verifier produces a **hierarchical reward**: hard gates first, then deterministic structural checks, design-system adherence, visual judgments, and accessibility heuristics, weighted per task. Each check is scoped to defeat a specific reward-hacking pattern.

The same machinery scales to hundreds of tasks via our **Figma Task Builder** plugin, where subject matter experts convert live Figma frames into runnable tasks in minutes.

2. Why Figma is the Right Surface for Design RL

Figma is the de facto enterprise design surface, which makes capability gains here directly transferable to the workflows researchers care about (design-to-code, generative UI, design system maintenance):

- **~13 million monthly active users** across designers, PMs, engineers, and marketers, with two-thirds of users now non-designers — turning Figma into a cross-functional design surface, not just a designer tool ([Kleiner Perkins](#), [AlInvest analysis](#)).
- **78% of the Fortune 2000** and roughly **95% of the Fortune 500** are paying customers, and Figma is in **~98% of organizations** that buy design software in 2026 ([SaaStr learnings from S-1](#), [Longyield analysis](#), [Ramp vendor data](#)).
- **Q1 2026 revenue of \$333M (+46% YoY)** with full-year guidance raised to **~\$1.42–\$1.43B**, and a **net dollar retention of 139%** — i.e., enterprises are not just adopting Figma, they are deepening usage, particularly around AI-assisted design ([Figma IR](#), [Q1 2026 transcript](#)).

- **MCP is the access layer for AI agents.** Figma now ships first-party Dev Mode and Remote MCP servers integrated with Claude Code, Codex, Cursor, and Gemini CLI. The Figma file is rapidly becoming a structured tool target for agents, not a static asset ([Figma Dev Mode MCP announcement](#), [Figma developer docs](#)).

Content from third-party sources was rephrased and summarized for licensing compliance; figures remain attributed inline.

The implication for RL: training on Figma-shaped tasks puts an agent on the same surface its real users will operate on, with the same MCP tool vocabulary, the same node graph semantics, and the same component/variable abstractions.

3. The Figma Clone

A central pillar of this environment is that it does **not depend on Figma**. We ship our own Figma-shaped runtime, the **Figma Clone**, which lets a researcher spin up thousands of training trials in parallel on commodity Linux containers without authenticating against Figma, without managing teams, without rate limits, and without any cloud dependency.

3.1 Why a clone instead of driving real Figma

Driving real Figma for RL training is operationally untenable:

Real Figma	Our Figma Clone
Cloud-only, requires team & seat management	Local container, zero accounts
Strict per-account rate limits	Determined by your CPU
Plugin sandbox is single-process, single-window	Headless, parallelizable
State leaks across runs	Each trial gets a fresh, baked design
Network and login flakiness	Hermetic
Hard to snapshot/diff for graders	Single canonical document per file

For training, the requirements are not "be Figma" — they are: **expose the same agent surface area** (Plugin API + MCP tools), **accept real Figma designs**, and **render outputs that we can deterministically compare**. The Figma Clone was scoped exactly to those goals, with everything else explicitly out of scope (multiplayer, prototyping, FigJam, video, Code Connect, the canvas editor itself).

3.2 Capabilities (what agents see and use)

At the surface level, the Figma Clone provides:

- **Figma Plugin API parity.** Agents can run `use_figma { code, description }` containing JavaScript written against the Figma Plugin API — `figma.createFrame`, auto-layout properties, variables, styles, component instances, traversal, fill/stroke/effect/grid styles, bound variables, styled text segments, and so on. This means the **same agent prompt and same code patterns** work against either Figma Desktop or our clone, and any prompt corpus or skill we build is portable.
- **Figma MCP interface.** The Figma Clone speaks Streamable HTTP MCP. The toolset (`get_metadata`, `get_screenshot`, `get_design_context`, `get_variable_defs`, `search_design_system`, `use_figma`, `create_new_file`, `open_file`, ...) and tool descriptions track Figma's published MCP tools and prompts so agent behavior generalizes.
- **Deterministic, single-document I/O.** Every design is a structured envelope holding the document tree, components, text styles, paint styles, effect styles, grid styles, variables, variable collections, and inline asset bytes. The verifier diffs two envelopes (before/after) — this is what makes scoring stable across runs.
- **Real Figma fidelity on import.** We import frames, text (with styled segments), shapes, vectors, boolean operations, groups, sections, components, instances, slices, auto-layout, constraints, pattern fills, variables and bound fields, and paint/effect/text/grid styles. Coordinates are parent-relative, matching `node.x / node.y` semantics in the Plugin API.
- **Pixel rendering.** A built-in design compiler produces HTML/CSS plus Playwright-driven PNG screenshots per node, used both by `get_screenshot` for the agent and by the verifier's visual judges. We track parity against real Figma across **110 verification scenarios** spanning foundational and advanced Plugin API surfaces.
- **Fast response time.** A thin per-task image is layered on top of a prebuilt base image. New tasks build in seconds, not minutes. The MCP server starts inside the same container, so agents and graders share a hermetic, low-latency loop.
- **Isolation by default.** Sandboxed plugin code, no `fetch` and no remote `createImageAsync(url)` unless explicitly enabled, and no team-library or remote-key dependencies. Trials cannot phone home or cross-contaminate.

3.3 What we deliberately do not do

We are explicit about non-goals: the Figma Clone is not a Figma replacement. It excludes the canvas editor, multiplayer, FigJam, Slides, Buzz, prototyping, Dev Mode codegen plugins, Code Connect, video and embed nodes, plugin payments, plugin-data persistence on nodes, and byte-identical Figma rendering. This is a feature, not a limitation: it lets us guarantee the surface that *does* exist behaves consistently and is fully testable.

4. Task Categories This Environment Unlocks

Because the Figma Clone is a real Figma-shaped runtime with both an editable document and a renderer, this environment is not limited to a single task style. The same infrastructure cleanly supports the four high-value categories researchers are pursuing today:

Category	What the agent does	What the Figma Clone contributes
Figma design → code	Read structure with <code>get_design_context</code> / <code>get_metadata</code> , emit React/HTML/CSS or framework-specific code.	Faithful tree, variables, styles, auto-layout, and screenshots — the same context Figma's MCP gives agents. Code can be compiled and screenshot-compared back against the source design.
Code → Figma design	Given a component or page in code, produce the equivalent Figma file.	The agent emits <code>use_figma</code> Plugin API code; the Figma Clone executes it deterministically and the verifier diff-checks the resulting tree.
Screenshot → Figma design	Reconstruct a design from an image (mock, competitor screenshot, hand sketch).	Agent uploads the asset under <code>/app/assets/</code> , builds the tree via Plugin API, and the verifier scores both structural fidelity and visual similarity to the reference image.
Text → design	Generate a design from a brief or spec.	Agent creates frames/components/variables from scratch; rubrics measure design-system adherence, hierarchy, contrast, and task completeness via LLM-judge visual checks.

The same tasks also support **targeted edits**, **design-system implementation**, and **regression-style fixes** without changing the runtime — only the rubric.

5. Real Figma Designs as Task Sources

Synthetic stubs do not stress an agent the way a real production design system does. Our task sources are **real Figma files** that subject matter experts port into the Figma Clone via the **Local Figma MCP plugin** running inside Figma Desktop. The plugin captures a snapshot of the open file (frames, text with styled segments, shapes, vectors, components, instances, variables, bound variables, paint/effect/text/grid styles, auto-layout, pattern fills, embedded image bytes) and POSTs it to the Figma Clone's import endpoint, which converts it into a structured file envelope.

The result is a design that:

- Was authored in real Figma against real designer constraints.
- Carries the file's variables, styles, and components — not approximations.
- Is small enough to bake into a task image and diff in milliseconds.
- Can be re-rendered to PNG to drive visual checks.

Tasks are seeded from a curated library of these designs. One of the bundled tasks, for example, is built from a real marketing site design imported this way and asks the agent to extend its footer with terms and privacy links — exactly the kind of incremental, contextual edit that exercises both structural understanding and design-system fluency.

6. Task Variety

The environment supports a wide spectrum of task shapes out of the box. All of them share the same runtime, MCP surface, and verifier — only the instruction and `eval-spec.json` change. The bundled **oker** task set (ten tasks seeded from a real e-commerce design system) illustrates the range:

- **Greenfield screen builds.** Create a full mobile screen from scratch inside an existing production file — e.g. an **Order Details** screen with per-shipment tracking, status-dependent content (Ordered, Shipped, Out for Delivery, Delivered), item thumbnails, amount breakup, and delivery address; a **Chat Support** screen with order context bar, user/agent message bubbles, image attachments, and session controls; or a **max OTP attempts** cooldown state in an onboarding sequence (disabled resend button plus countdown copy). Exercises navigation of a large file, reuse of existing components and assets, auto-layout at screen scale, and multi-section information hierarchy — graded with `task_completeness`, `good_design`, and reference-backed consistency checks.
- **Targeted edits within existing screens.** Restructure a complex screen without rebuilding it — e.g. on the **PDP** frame, insert a shipping-and-delivery section below color/size choices, move Add to Cart into that section, expand Product Details by default, reorder More Information and "See how the product is styled," and remove duplicate CTAs. Changes are confined via `allowed_change_inside_ids`; structural checks assert real modification under the target frame, not cosmetic tweaks elsewhere.
- **Contextual extensions to existing flows.** Add missing UI that must match sibling patterns in the same file — e.g. a **Brand Filter** screen for the PLP filter flow (multi-select brands with icons, Select All / Deselect All, bottom Apply/Reset bar, left filter-type rail); or a **Sale** section on the home Shop tab (horizontal bundle carousel, urgency banner with remaining-count, eye-catching discount treatment, mixed product types per bundle). Graded structurally (required text, images, minimum additions) and visually via `design_consistency` criteria that pin layout to reference screens (e.g. brand listing positioned like the existing color filter).
- **Design-system implementation.** Systematic token and style work on real nodes — e.g. in the PDP frame, walk every descendant at any nesting level, create a named Fill/paint style for each fill (referencing existing color variables where applicable), and bind each node to its new style instead of inlining hex values. Graded heavily on style/variable-reuse and edited-regression checks, with a `design_consistency` rubric that requires the screen to look **pixel-identical** to the baseline despite the binding changes.
- **Multi-screen and multi-state flows.** Tasks that span several frames or states in one edit — e.g. **Share Collection** (collection page with share affordance → bottom sheet with searchable friend list, checkboxes, and Share CTA, using assets from `/app/assets/`); **My Orders** plus **Order Details** as companion screens in a new Orders section; or adding a new cooldown state to an existing OTP onboarding sequence without disturbing prior states. The verifier scopes gates and checks per parent so credit and penalties stay localized to the frames the agent touched.
- **Reference-driven variants.** Produce an alternative design that must diverge in specified ways while staying on-system — e.g. a **Browse/Category variant** that replaces the small-icon list with a two-column grid of large image-backed category cards and legible text overlays, without modifying the original

screen (`additions_only`). Scored via `design_preference` against author reference PNGs and `design_consistency` criteria on typography, spacing, radii, and icon treatment.

- **Regression-style preservation.** Edits where the visible design must not change even though the document model does — most clearly the apply-styles task above, where inlining fills into shared styles must not alter rendered appearance. The edited-regression and style-reuse checks catch agents that strip `textStyleId` / `fillStyleId` bindings or substitute one-off values; visual judges confirm the before/after screenshots still match.

Per-task `category_importance` lets subject matter experts tune what matters: apply-styles upweights `metadata` (diff judge) and `design_system`; screen-building tasks upweight `visual`; confined PDP edits lean on gates plus `task_completeness`. Optional checks let the author reward best-effort behavior without making the gate fail.

7. Rubrics — What We Check and Why

The verifier is a Python evaluation engine that runs in-process inside the trial container under RewardKit. Each rubric category exists because it defeats a specific reward-hacking failure mode we have observed in agents.

7.1 Hard Gates

Gates encode the structural invariants of the task. **Failing a gate caps the final reward at 20% of raw**, which is what makes it a *gate* and not a soft check.

- **Require change** — the design must actually differ from the baseline. *Prevents:* agents that "complete" the task by saying they did and never invoking `use_figma`.
- **Preserve specified nodes** — listed nodes must still exist in the result. *Prevents:* agents that delete or recreate the file to satisfy a check, breaking the surrounding design.
- **Confine changes to allowed regions** — all additions, modifications, and deletions must happen under specified roots. *Prevents:* drive-by edits to unrelated parts of the design (a frequent failure mode in long-context plans).
- **Additions only** — no modifications or deletions. *Prevents:* destructive shortcuts on tasks where the goal is purely additive.
- **No detached nodes** — every node created via `use_figma` must be attached to the document tree (recorded in `issues.hfc.json`). *Prevents:* agents that create nodes in memory but never parent them, leaving invisible or unreachable content.

7.2 Structural Checks

These are deterministic, programmatic, and configured per task. **Failing any required structural check caps the reward at 30% of raw.** Each check has a typed spec and validates against a published JSON schema.

- **Must contain text (scoped to a node or to new frames).** Asserts a substring appears in a target node, or in any frame the agent added. *Prevents:* the agent claims it added "Privacy Policy" but actually added "Privacy P" or wrote it in a hidden text node off-screen. Scoping to new frames prevents satisfying the check by editing pre-existing copy.
- **Must contain image.** A specific image (optionally hash-pinned) must appear in scope. *Prevents:* fabricating image presence with a colored rectangle or substituting a different asset.
- **Minimum additions or modifications under a parent.** At least N new (or modified) descendants under a parent. *Prevents:* skipping additions, or making cosmetic property tweaks that look like work but do nothing structural.
- **Component instances under a scope.** Asserts N instances of a specific component under a given scope. *Prevents:* hand-building a fake "Card" out of frames instead of actually instantiating the design system component — a very common shortcut.
- **Metadata-only changes under a parent.** All changes under a parent must be metadata (name, description, locked, exportSettings, reactions). *Prevents:* rename tasks where the agent rebuilds nodes instead of just renaming, or sneaks in geometry edits.
- **Exact property equality on a node.** For example, `name == "MainBoard"`. *Prevents:* "close enough" renames and casing drift.

7.3 Design-System Adherence

Three checks that fire automatically (no per-task config required), scoped to the **content nodes the agent touched** so unrelated parts of the file are not penalized.

- **Token adherence.** Every value the agent writes for a known role (text color, fill, stroke, font size, spacing, padding, stroke weight, corner radius, etc.) must come from the design's observed allowlist of tokens for that role. *Prevents:* the agent introducing a one-off `#3F51B5` when the design system has a `Primary/500` variable; introducing inconsistent paddings; using arbitrary font sizes.
- **Style and variable reuse.** When a value matches a bindable style or variable, the agent should bind to it (`textStyleId`, `fillStyleId`, `strokeStyleId`, `effectStyleId`, `gridStyleId`, bound variables) instead of inlining. *Prevents:* the agent stripping bindings to "simplify" the node — a regression that destroys a design system over time.
- **Edited regression.** A node that had a style/variable binding before the edit must still have one after. *Prevents:* the most insidious failure mode — the agent edits a node's text, accidentally drops `textStyleId`, and the design silently de-tokenizes.

These three checks together close the "looks right, breaks the system" loophole: a screenshot-based judge alone cannot tell that a binding was stripped.

7.4 Visual Checks (LLM Judge)

Visual rubrics use a vision-capable LLM (configurable via `EVAL_JUDGE_MODEL`, default `gemini/gemini-3.1-pro-preview`) and the Figma Clone renderer to grade what the eye actually sees. Five spec types in `eval-`

`spec.json`, each with a type-specific default weight inside the visual category (overridable per check via `weight`):

Type	Default weight	What it judges
<code>task_completeness</code>	8.0	Whether each task requirement is present <i>and</i> production-ready (not stubs or broken layout)
<code>design_preference</code>	3.0	Whether the agent output is closer to a reference mock than the baseline
<code>design_consistency</code>	2.0	How well the agent output matches a reference image against author-listed criteria (1–5 per criterion)
<code>design_fit</code>	2.0	Custom prompt on a specific node, comparing before/after screenshots for integration into the parent frame
<code>good_design</code>	1.0	General quality of the agent's new work — defect dimensions (placeholders, proportions, completeness) plus typography, spacing, color, hierarchy

Screenshot targets are resolved from a per-task `screenshot` block (`strategy`: `auto`, `explicit`, `largest_added_frame`, `minimal_enclosing`, `all_added_frames`, etc.). Renders land under `/logs/verifier/screenshots/`.

LLM-judge calls are parallelized (default 4) and can be skipped entirely (`skip_llm`) for fast offline iteration; skipped visual checks return a fixed placeholder score of **0.75** so authors can still iterate on structural rubrics.

7.5 Metadata Checks (LLM Judge, No Screenshots)

The `metadata_checks` array holds text-only LLM rubrics that share the `metadata` scoring category:

- `diff` — passes a structured edit-graph summary plus the task instruction to the judge; returns a 0–10 score. *Use*: low-cost signal on whether the structural delta matches the brief before paying for renders, or as a complement to visual checks.

Skipped metadata checks use the same **0.75** placeholder as visual checks.

7.6 Command Correctness

Always evaluated when `issues.hfc.json` records `use_figma` command attempts:

- **Command correctness** — fraction of recorded plugin runs that succeeded. If any run failed, the score is capped at **0.5** even when most runs succeeded, so flaky scripting cannot fully mask broken edits. When no commands were recorded, the subcheck is marked not applicable and dropped from the category average.

7.7 Heuristics

Always-on, deterministic, scoped to content the agent touched. Cheap signals that catch the long tail of "looks plausible to a judge, fails on inspection":

- **WCAG AA contrast.** Text foreground vs. resolved background luminance; score is `min(1, mean_ratio / 4.5)`. *Prevents:* legal-risk dark-on-dark or light-on-light text.
- **Distinct font count.** Penalizes more than five distinct font signatures. *Prevents:* "novelty by font soup," a typical generative failure mode.
- **Readable font size.** Penalizes any text below 10px. *Prevents:* the agent shrinking text to fit a constraint.

7.8 Why scoping matters (anti-reward-hacking note)

A subtle but critical design choice: **design-system checks and heuristics only consider the node IDs the agent touched** (additions and non-metadata modifications), traversed transitively. This means:

- An agent cannot lose a point because the *baseline* design has a low-contrast section unrelated to the task.
- An agent cannot earn a point by *fixing* an unrelated low-contrast section instead of doing the task.
- A "rename a node" task does not get scored for typography variety, because the rename is metadata and produces no content scope.

8. Hierarchical Rewards

The verifier runs inside the trial container under **RewardKit** and exposes a single criterion, `figma_design_score`, in `[0, 1]`. Harbor uses that value as the trial reward. The same run also writes detailed reports where the score appears on a 0–10 scale:

```
reward = clamp(0, 1, score_0_10 / 10)
score_0_10 = clamp(0, 10, completion_gate × raw × 10)
```

Scoring has two layers that must not be conflated:

1. **completion_gate** — multiplicative hard caps from gates and required structural checks. **Gates never enter the quality average.**
2. **raw** — weighted mean over **scoring categories only**, each category itself a weighted mean of its applicable subchecks.

```
completion_gate = 1.0
→ min(0.2) if any applicable gate scores below 1.0
→ min(0.3) if any applicable required structural check scores below 1.0

raw = Σ (category_mean × normalized_importance)
    over scoring categories with at least one applicable subcheck

category_mean = Σ (subcheck_score × subcheck_weight) / Σ subcheck_weight
               (not-applicable subchecks omitted)
```

Scoring categories and default importance

These six categories contribute to `raw`. Per-task overrides go in `eval-spec.json` under `category_importance` (the legacy `weights` key is also accepted; a `gates` entry is ignored if present). Importances are normalized to sum to 1 among categories that actually apply.

Category	Default importance	Subchecks
<code>commands</code>	0.15	<code>use_figma</code> success rate from <code>issues.hfc.json</code>
<code>checks</code>	0.35	Per-task structural checks (<code>must_contain_text</code> , <code>property_on_node</code> , ...)
<code>design_system</code>	0.2	Token adherence, style/variable reuse, edited regression
<code>visual</code>	0.35	LLM + screenshot judges (<code>good_design</code> , <code>task_completeness</code> , ...)
<code>heuristics</code>	0.1	Contrast, font count, readable font size
<code>metadata</code>	0.1	Text-only LLM judges (<code>diff</code>)

Within the **visual** category, each check type has a built-in default weight (e.g. `task_completeness` **8.0**, `design_preference` **3.0**, `design_consistency` **2.0**, `design_fit` **2.0**, `good_design` **1.0**). Authors can override any subcheck via an explicit `weight` field in the spec entry.

Gates (`require_change` , `preserve_ids` , `allowed_change_inside_ids` , `additions_only` , `no_detached_nodes`) appear in the report under the `gates` category for transparency but affect only `completion_gate` .

Example

A task might set `"category_importance": { "visual": 0.51, "checks": 0.35, "commands": 0.15, ... }` to emphasize visual judgment on a screen-building task, while a rename task might zero out visual importance and lean on `checks` .

Reports and RL properties

After each trial, the verifier writes:

- `/logs/verifier/reward.json` — final `RewardKit` reward
- `/logs/verifier/eval-report.json` — `score` , `completion_gate` , `raw` , and per-subcheck list
- `/logs/verifier/eval-report-details.json` — category-grouped breakdown with every subcheck's score (as `reward`), weight, applicability, and judge details

This shape gives RL several useful properties:

- Sharp gates, dense gradient.** An agent that only half-completes a task can still earn a non-zero `raw` score to climb, but `completion_gate` prevents reaching the top band without passing gates and required structural checks — the same pattern that stabilizes RL on multi-step tool-use tasks.
- Per-subcheck breakdown.** Matched node IDs, contrast ratios, command success counts, and raw judge JSON are all available for offline rejection sampling, dense reward shaping, or post-hoc analysis.

- **Determinism where it matters.** Gates, structural checks, design-system checks, heuristics, and command correctness are fully deterministic. **Visual** and **metadata** categories use an LLM judge at temperature 0 with structured JSON output, parsed strictly. Set `skip_llm` to bypass LLM calls during rubric development (placeholder score **0.75**).
-

9. Subject-Matter-Expert-Authored Tasks and Per-Task Rubrics

This is not a benchmark dataset. Every task and every rubric is **authored by a subject matter expert** — a designer who can read the design, understand the intent, and configure the verifier accordingly. The artifact that captures their judgment is `eval-spec.json`, validated against a published JSON schema.

What a subject matter expert configures per task:

- **Hard gates** appropriate to the task shape (require change, preserve specified nodes, confine changes to allowed regions, additions only).
- **Structural checks** with target node IDs and scopes pulled directly from the source Figma file.
- **Visual checks** — which spec type, which node to render, which criteria to weight (consistency vs. fit), and any reference asset.
- **Design-system stance** — opting a creative task out of token-allowlist enforcement; default behavior enforces the system.
- `category_importance` — to express "this task is a rename, structural correctness is everything" vs. "this is a hero section, visual judgment dominates."
- **Visual and metadata checks** — which LLM judge types to run, screenshot strategy, reference assets, and per-check weights.
- **Command and gate stance** — whether `no_detached_nodes` or command correctness should factor into scoring for script-heavy tasks.

This per-task rubric authoring is what separates this from generic "is the JSON valid" evals. The grader has a **task-specific theory of correctness**, and that theory was written by a human who understands design.

10. Scalability — Figma Task Builder

To deliver hundreds of tasks at quality, we built a dedicated authoring tool. The **Figma Task Builder** is a Figma plugin plus companion HTTP service that lets a subject matter expert, while looking at a real Figma file, produce a finished task in a few clicks.

The flow:

1. The author opens a real design in Figma Desktop.
2. The **Local Figma MCP** plugin (our plugin, running in Desktop) shows a Task Builder tab.

3. The author picks the frames that are part of the task, optionally excludes nodes, writes the instruction, and fills the `eval-spec.json` with the help of a check catalog.
4. The plugin POSTs the snapshot to the Figma Clone's import endpoint to convert it into a structured file envelope.
5. The Task Builder service writes a draft and, on **Finalize**, promotes it to a runnable task — complete with task config, instruction, baked design, asset directory, and copied verifier.

This pipeline is what makes the contract economically realistic. Authors are not editing JSON in a text editor or wiring up Dockerfiles; they are working *inside Figma* with guardrails that ensure every task lands as a runnable, typed task.